If It Looks Like a Rootkit and Deceives Like a Rootkit

A Critical Examination of Kernel-Level Anti-Cheat Systems

Christoph Dorner, Lukas Daniel Klausner. *University of Applied Sciences St. Pölten*
Leonore Juif, Lou Tinevez. *University of Rennes*

03-03-2026

Summary

1 Introduction: KLAC and rootkits

2 Methodology: The 7 Metrics of "Rootkit-ness"

3 Case Studies: The "Four Horsemen" of Anti-Cheat

4 Analysis & Results: Who is a "Rootkit"?

5 Privacy and Ethical Concerns & Discussion

Introduction to Kernel Level Anti-cheat and rootkits

Context : The Cheating Arms Race

Online gaming is a multi-billion dollar industry. Cheating (**aimbots**, **wallhacks**) ruins the experience and revenue.

Cheats moved from "**User-Mode**" (Ring 3) to the "**Kernel**" (Ring 0) to bypass standard detection. (cf. fig 1)

→ **The Solution**: Developers responded with **Kernel-Level Anti-Cheat** (KLAC) to gain the same high-level privileges as the cheats.

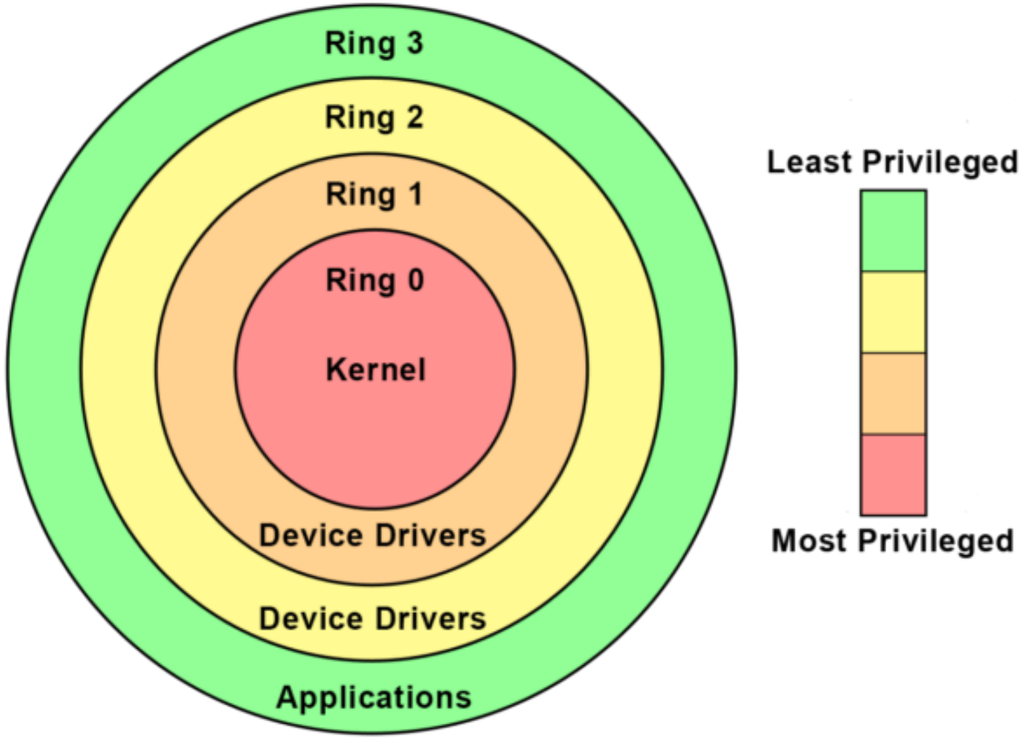
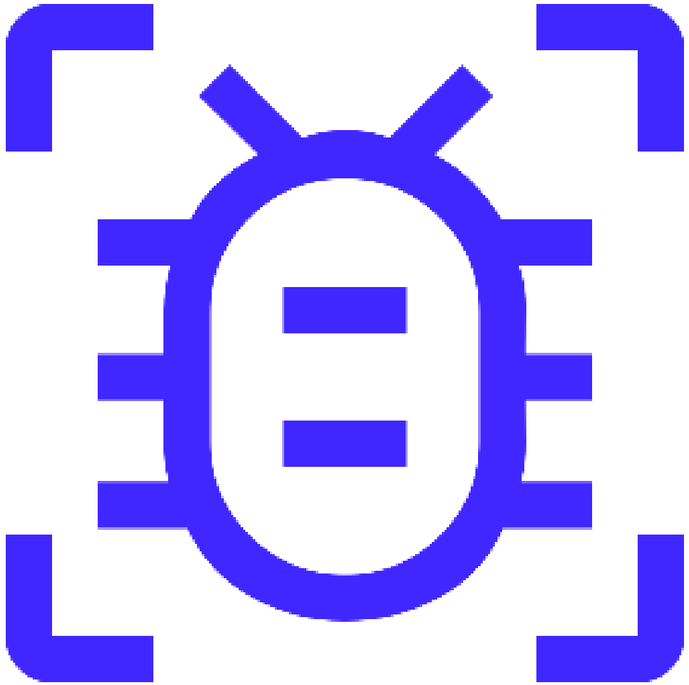


Figure 1 – Kernel Levels

What is a Rootkit?

Definition : Software designed to provide administrative access while hiding its presence.



Key Characteristics:

- **Persistence** (starts with the OS).
- **Stealth** (hides files, processes, and network connections).
- **Privilege** (runs in the kernel, having total control).

→ Many **KLAC** systems use the exact same techniques as rootkits to maintain integrity.

Methodology: The 7 Metrics of "Rootkit-ness"

Metrics of "Rootkit-ness"

We developed **7 metrics** to evaluate software for rootkit-like behavior:

- **Evasion**: Does it hide its presence or hooks from the OS?
- **Virtualization**: Does it use obfuscation or VM-based protection?
- **Execution Time**: Does it load at boot (before the user or even the OS is ready)?
- **Remote Access**: Can the developer control/update it remotely without user input?
- **Information Exfiltration**: Does it scan and send back broad system data?
- **Network Manipulation**: Does it hook into the network stack?
- **Removability**: How difficult is it to fully uninstall?

Case Studies: The "Four Horsemen" of Anti-Cheat

Case Studies

We examines four industry leaders:

BattlEye

Used in PUBG, Rainbow Six Siege.
Known for **memory scanning**.

Easy Anti-Cheat (EAC)

Owned by Epic Games (Fortnite, Apex Legends). Focuses on **integrity checks**.

FACEIT AC

Used in Counter Strike Global Offensive
High-level competitive **platform driver**.

Riot Vanguard

The **most controversial** system, used in Valorant and other Riot Games.
The focus of our study.

Analysis & Results: Who is a "Rootkit"?

Analysis

Feature	EAC / BattlEye	FACEIT AC	Riot Vanguard
Persistence	Low (Game-time only)	Medium (System-wide)	High (Boot-time)
System Control	Game-specific	System Integrity	Hardware Overrule
User Transparency	High	Medium	Low
Primary Goal	Detect Cheats through Memory or Integrity checks	Ensure Clean Boot	Total System Lockdown

Results

The Threshold: We set a "Rootkit-likeness" score (**R**).

Findings:

- **BattlEye & EAC**: **Scored lower**. They generally only run when the game is active.
- **Vanguard & FACEIT**: Scored **R ≥ 4** (High).

→ Vanguard Analysis: It loads at **system boot**, remains active even when the game is **closed**, and has **aggressive self-protection** that can block other **legitimate drivers**.

Privacy and Ethical Concerns

Privacy and Ethical Concerns

Still, there are serious privacy concerns that come with granting such power to a third-party application.

System Stability

Kernel drivers can cause "Blue Screens of Death" (BSOD) if they **conflict** with other hardware.

Privacy

These systems have the technical capability to read any file or monitor any keystroke (even **passwords** for **banking**) because they sit at the **highest privilege level**.

Security Risk

If a KLAC driver has a vulnerability, it becomes a "**backdoor**" for actual hackers to **take over the entire PC**.

Discussion: The "Necessary Evil"?

Developer Perspective:

Without kernel access, competitive integrity is **impossible**.

User Perspective:

Users are forced to trade system **sovereignty** for the ability to play a game.

Alternative

We could look into server-side **AI detection** as a **less intrusive** path forward.

Conclusion:

Vanguard and **FACEIT** exhibit **clear rootkit-like properties**.

→ We need a **standardized** "Cybersecurity for Anti-Cheat" framework to ensure these tools protect games without compromising **user privacy** or **system safety**.

Questions ?

Thank you!

References

- **Research Paper**
  https://arxiv.org/abs/2408.00500

- **Icons**
  https://fonts.google.com